SERIES 04

github.com/madhavan-21/ebpf-mastery

Uprobe-Based SSL/TLS Plaintext Interception

Lesson 04: Intercepting Encrypted HTTPS Traffic without Certificate Injection

Madhavan S
SDE - Atatus

Date: July 6, 2026
Version: v1.0.0

Table of Contents

Q2. Why can we read plaintext data from SSL_write() even though the traffic is encrypted on the wire?

2. Intermediate Level

   Q3. How does bpf_probe_read_user() differ from bpf_probe_read()? Why is the distinction critical for uprobes?

   Q4. How do you find the exact byte offset of SSL_write inside libssl.so to attach a uprobe?

   Q5. Explain what a Uretprobe is and why it is needed for SSL_read() but not for SSL_write().

3. Advanced Level

   Q6. SSL_write is different across OpenSSL 1.x and 3.x. How would you make your uprobe binary-portable across versions?

4. System Design & Scenario-Based

   Q7. Design a production zero-code HTTP request tracer that captures request/response pairs for all services in a Kubernetes cluster without modifying any application.

5. Troubleshooting & Debugging

   Q8. Your uprobe is attached but captures zero events, even though curl https://example.com is clearly making TLS connections. List 4 likely causes.

Hands-on Lab & Homework Assignments

1. Practical Exercises & Research Tasks

   Task 1: Find SSL_write in the ELF Binary

   Task 2: Inspect the Uprobe with bpftool

2. Coding Assignment: Capture SSL_read (Decrypted Responses)

   Requirements:

4. Advanced Challenge: Multi-Process TLS Monitor Dashboard

Lesson 04: Uprobe-Based SSL/TLS Plaintext Interception

1. Lesson Overview

In this lesson, you will build a **TLS plaintext interceptor** — the same technique used by Pixie, Cilium, and Datadog to capture encrypted HTTPS request/response bodies **without touching certificates, proxies, or application code**. We will attach eBPF **uprobes** directly to the `SSL_write` and `SSL_read` functions inside `libssl.so`, capturing the unencrypted application payload at the exact moment it passes through the OpenSSL library — before encryption on write, and after decryption on read.

This is how modern APM tools achieve "zero-code" HTTP request tracing even for TLS-encrypted services.

---

2. Foundations & Core Technical Concepts

What is SSL/TLS?

TLS (Transport Layer Security) is the cryptographic protocol that encrypts HTTP traffic (HTTPS). When your browser connects to `https://google.com`:

1. A TLS handshake establishes a shared symmetric encryption key.
2. All subsequent HTTP request/response data is encrypted with that key using AES-GCM.
3. The encrypted ciphertext is sent over TCP.

At the network layer, all you see is random-looking bytes. `tcpdump` and Wireshark show nothing useful for HTTPS traffic. Traditional network-based APM tools are completely blind to the application payload.

What is OpenSSL and libssl.so?

OpenSSL is the most widely used TLS library on Linux. Applications that want to make HTTPS connections call OpenSSL functions:

- `SSL_write(ssl, buf, len)` — encrypts `buf` (plaintext) and sends it over the network.
- `SSL_read(ssl, buf, len)` — receives encrypted data from the network and decrypts it into `buf`.

The critical insight: Between the application calling `SSL_write(buf)` and OpenSSL encrypting `buf`, the data exists in plaintext in user-space memory. If we can hook into `SSL_write` at that exact moment, we capture the plaintext before it's ever encrypted.

What are Uprobes?

Uprobes (User-space Probes) are the user-space equivalent of kprobes. They allow eBPF programs to attach to specific functions inside user-space executables and shared libraries.

- **How they work**: When you attach a uprobe to `SSL_write` in `libssl.so`, the kernel:
  1. Finds the virtual address of `SSL_write` in the shared library's ELF symbol table.
  2. Patches the first instruction of `SSL_write` with an `int3` breakpoint.
  3. When any process calls `SSL_write`, the breakpoint fires your eBPF program.
- **Uretprobes**: Fire when the function returns. Used to capture `SSL_read` output (the decrypted data is in the return buffer).

Why Can't We Just Use tcpdump?

Approach	Can See HTTPS?	Requires Root?	App Modification?	Overhead
`tcpdump`	No (encrypted)	Yes	No	Low
Man-in-the-Middle proxy	Yes (via cert injection)	No	**Yes** (trust store)	High
`strace` (ptrace)	Yes	Yes	No	**100–1000x slowdown**
eBPF Uprobe	**Yes**	**Yes**	**No**	**<1%**

What is PT_REGS_PARM?

When a function is called on x86_64, arguments are passed in CPU registers following the **System V AMD64 ABI**:

- Argument 1 → `%rdi`

- Argument 2 → `%rsi`
- Argument 3 → `%rdx`
- Argument 4 → `%rcx`

In a uprobe, `struct pt_regs *ctx` contains a snapshot of all CPU registers at the moment of the probe. `PT_REGS_PARM1(ctx)` extracts `%rdi` (the first argument). For `SSL_write(SSL *ssl, const void *buf, int num)`:

- `PT_REGS_PARM1(ctx)` = the `SSL *ssl` connection handle
- `PT_REGS_PARM2(ctx)` = the `const void *buf` plaintext buffer pointer
- `PT_REGS_PARM3(ctx)` = the `int num` length

What is bpf_probe_read_user?

Since `buf` is a pointer into user-space virtual memory (not kernel memory), we cannot dereference it directly inside a BPF program. We must use `bpf_probe_read_user()` which safely copies bytes from user-space memory into our BPF stack buffer, handling page faults gracefully.

---

3. Problem Statement & Real-World Use Case

The Problem: Zero-Visibility into Encrypted Microservices

A microservices architecture running 200 services, all communicating over HTTPS/mTLS. A latency spike appears in service B. The traces show service A is calling service B, but you cannot see:

- What the HTTP request URL was.
- What the request body contained.
- What the response status code was.
- How long the TLS handshake took.

Traditional APM agents require you to instrument your code with SDK calls. But if you have 200 services in 6 different languages, that's an enormous engineering investment — and it still misses third-party libraries.

Real Production Story: How Pixie Works

Pixie (acquired by New Relic, now open-source under CNCF) is a Kubernetes-native observability platform that provides automatic full-body HTTP request tracing with zero code changes. Its entire foundation is eBPF uprobe-based TLS interception.

Pixie deploys a DaemonSet that:

1. Watches for new container processes starting.
2. Dynamically attaches uprobes to `SSL_write` / `SSL_read` in each process's `libssl.so`.
3. Streams decrypted HTTP payloads to its in-cluster analytics engine.
4. Presents full request/response tracing in its UI — showing request headers, body, response status, and latency — for every HTTPS call across the cluster.

Cilium uses the same technique in its Hubble network observability layer for L7 protocol visibility. **Datadog** uses it for their Continuous Profiler and Network Performance Monitoring products.

---

4. How OpenSSL Symbol Discovery Works

Before attaching the uprobe, our loader needs to find the exact virtual address of `SSL_write` in `libssl.so`. This is done by parsing the ELF symbol table:

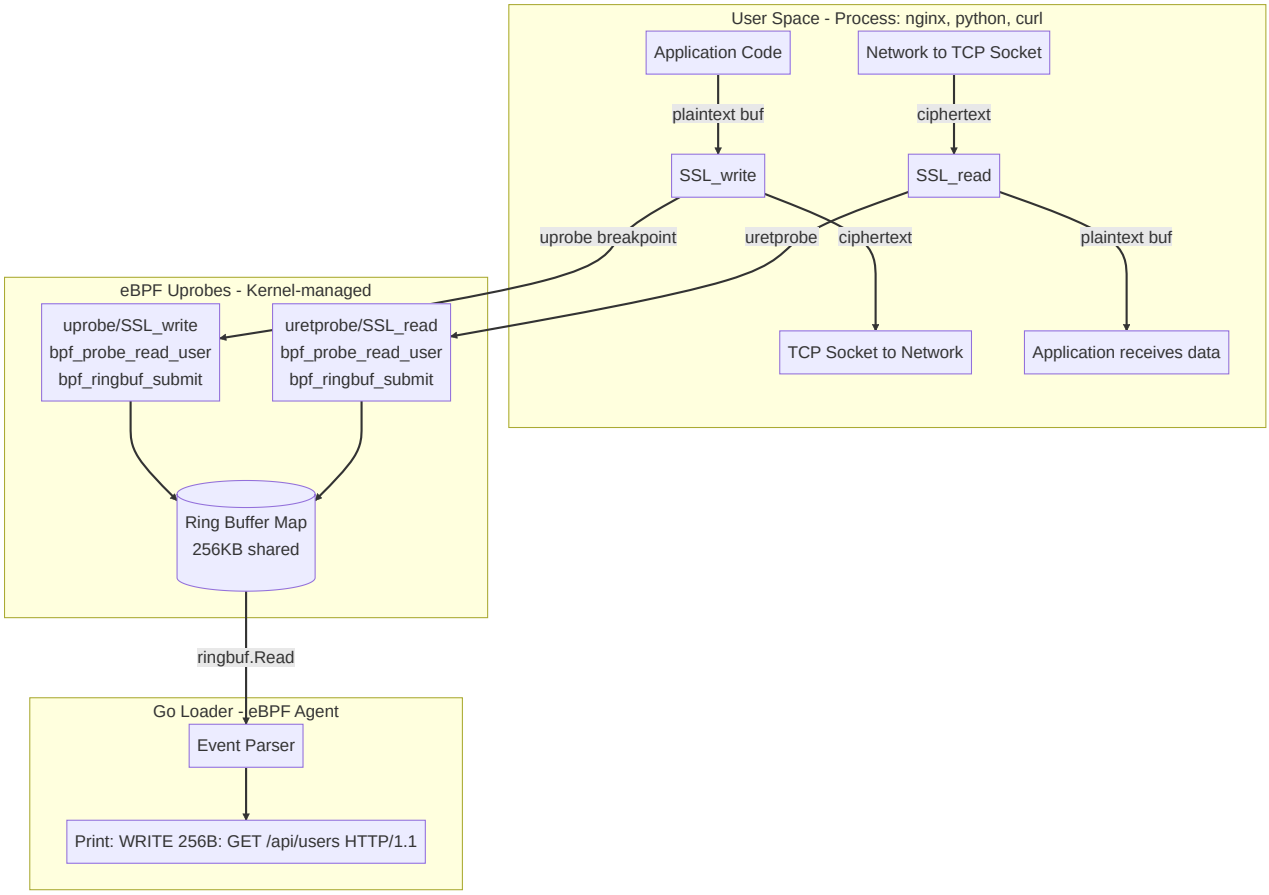
```
# Find the libssl.so path
ldconfig -p | grep libssl
# Output: libssl.so.3 (libc6,x86-64) ⇒ /usr/lib/x86_64-linux-gnu/libssl.so.3

# Find SSL_write's offset within the library
nm -D /usr/lib/x86_64-linux-gnu/libssl.so.3 | grep SSL_write
# Output: 0000000000045c30 T SSL_write
```

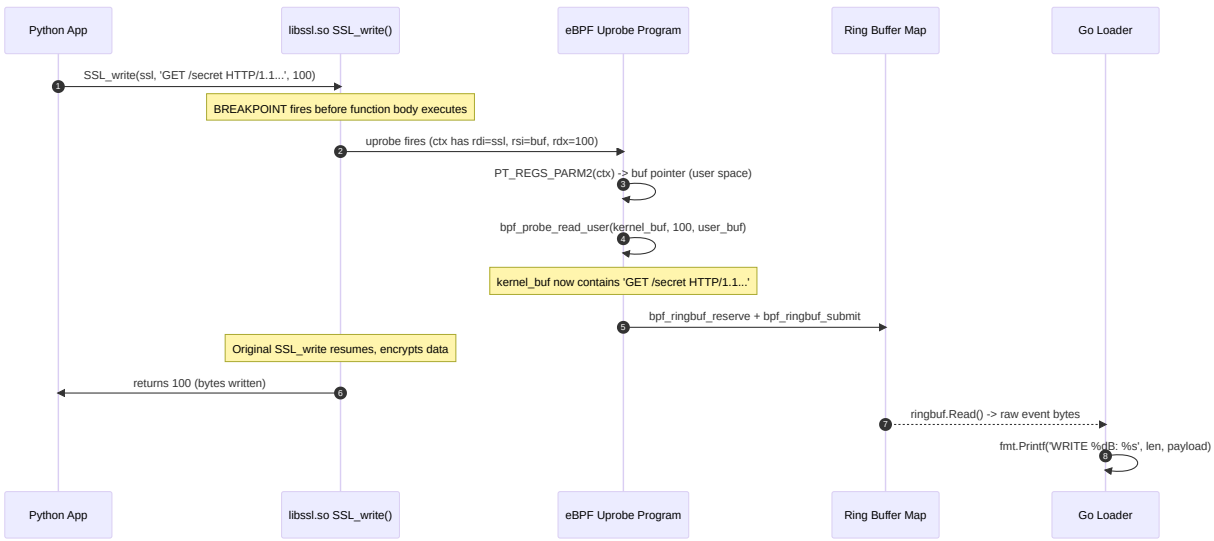
The `cilium/ebpf` Go library handles this automatically via `link.Uprobe(libPath, "SSL_write", prog, nil)` — it parses the ELF file and finds the symbol offset.

---

5. Architecture Diagram



6. SSL_write Call Sequence



7. Step-by-Step Code Walkthrough

The eBPF C Program (`main.bpf.c`)

```c
#include <vmlinux.h>
#include <bpf/bpf_helpers.h>

#define MAX_PAYLOAD 512

// Structure sent to user-space for each SSL event
struct ssl_event_t {
    __u32 pid;
    __u8  direction;    // 0 = SSL_write (outbound), 1 = SSL_read (inbound)
    __u32 len;
    char  payload[MAX_PAYLOAD];
};

// Ring Buffer — single shared buffer for all CPUs (modern alternative to perf array
struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 256 * 1024);   // 256KB total
} ssl_events SEC(".maps");

// Temporary storage for SSL_read: save the user buffer pointer on entry
// so we can read it on return (when decrypted data is populated)
struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 1024);
    __type(key, __u32);    // PID
    __type(value, __u64);  // User-space buffer pointer
} ssl_read_args SEC(".maps");

// Fires BEFORE SSL_write executes — plaintext is in buf (arg 2)
SEC("uprobe/SSL_write")
int trace_ssl_write(struct pt_regs *ctx) {
    const void *buf = (const void *)PT_REGS_PARM2(ctx);   // plaintext buffer
    int len = (int)PT_REGS_PARM3(ctx);                     // length to write

    if (len <= 0) return 0;

    // Reserve space in the ring buffer (zero-copy direct write)
    struct ssl_event_t *event = bpf_ringbuf_reserve(&ssl_events, sizeof(*event), 0);
    if (!event) return 0;
```

```
    event->pid = bpf_get_current_pid_tgid() >> 32;
    event->direction = 0;                    // outbound
    event->len = len > MAX_PAYLOAD ? MAX_PAYLOAD : len;

    // Copy plaintext from user-space into ring buffer
    bpf_probe_read_user(event->payload, event->len, buf);

    bpf_ringbuf_submit(event, 0);
    return 0;
}

// Fires BEFORE SSL_read — save the output buffer pointer
// The buffer isn't populated yet (it's filled by the function body)
SEC("uprobe/SSL_read")
int trace_ssl_read_enter(struct pt_regs *ctx) {
    __u32 pid = bpf_get_current_pid_tgid() >> 32;
    __u64 buf_ptr = PT_REGS_PARM2(ctx);   // Save user buffer address
    bpf_map_update_elem(&ssl_read_args, &pid, &buf_ptr, BPF_ANY);
    return 0;
}

// Fires AFTER SSL_read returns — buffer is now filled with decrypted data
SEC("uretprobe/SSL_read")
int trace_ssl_read_exit(struct pt_regs *ctx) {
    __u32 pid = bpf_get_current_pid_tgid() >> 32;
    int ret = (int)PT_REGS_RC(ctx);   // Return value = number of bytes read

    if (ret <= 0) {
        bpf_map_delete_elem(&ssl_read_args, &pid);
        return 0;
    }

    // Retrieve the buffer pointer we saved on entry
    __u64 *buf_ptr = bpf_map_lookup_elem(&ssl_read_args, &pid);
    if (!buf_ptr) return 0;
    bpf_map_delete_elem(&ssl_read_args, &pid);

    struct ssl_event_t *event = bpf_ringbuf_reserve(&ssl_events, sizeof(*event), 0);
    if (!event) return 0;

    event->pid = pid;
    event->direction = 1;    // inbound
    event->len = ret > MAX_PAYLOAD ? MAX_PAYLOAD : ret;
    bpf_probe_read_user(event->payload, event->len, (void *)*buf_ptr);

    bpf_ringbuf_submit(event, 0);
    return 0;
```

```
}

char LICENSE[] SEC("license") = "GPL";
```

The Go Loader (`main.go`)

```go
package main

import (
    "bytes"
    "encoding/binary"
    "fmt"
    "log"
    "os"
    "os/signal"
    "syscall"

    "github.com/cilium/ebpf"
    "github.com/cilium/ebpf/link"
    "github.com/cilium/ebpf/ringbuf"
)

type SslEvent struct {
    PID       uint32
    Direction uint8
    _         [3]byte // C struct alignment padding
    Len       uint32
    Payload   [512]byte
}

func main() {
    spec, err := ebpf.LoadCollectionSpec("main.bpf.o")
    if err ≠ nil {
        log.Fatalf("load spec: %v", err)
    }

    var objs struct {
        ProgWrite      *ebpf.Program `ebpf:"trace_ssl_write"`
        ProgReadEnter  *ebpf.Program `ebpf:"trace_ssl_read_enter"`
        ProgReadExit   *ebpf.Program `ebpf:"trace_ssl_read_exit"`
        SslEvents      *ebpf.Map     `ebpf:"ssl_events"`
    }
    if err := spec.LoadAndAssign(&objs, nil); err ≠ nil {
        log.Fatalf("load objects: %v", err)
    }

    // Find the libssl path (adjust for your system)
    libPath := "/usr/lib/x86_64-linux-gnu/libssl.so.3"

    // Attach uprobe to SSL_write (fires on function ENTRY)
```

```
upWrite, err := link.Uprobe(libPath, "SSL_write", objs.ProgWrite, nil)
if err ≠ nil {
    log.Fatalf("uprobe SSL_write: %v", err)
}
defer upWrite.Close()

// Attach uprobe to SSL_read entry (to save the buffer pointer)
upReadEnter, err := link.Uprobe(libPath, "SSL_read", objs.ProgReadEnter, nil)
if err ≠ nil {
    log.Fatalf("uprobe SSL_read enter: %v", err)
}
defer upReadEnter.Close()

// Attach uretprobe to SSL_read exit (to read decrypted data)
upReadExit, err := link.Uretprobe(libPath, "SSL_read", objs.ProgReadExit, nil)
if err ≠ nil {
    log.Fatalf("uretprobe SSL_read: %v", err)
}
defer upReadExit.Close()

rd, err := ringbuf.NewReader(objs.SslEvents)
if err ≠ nil {
    log.Fatalf("ringbuf reader: %v", err)
}
defer rd.Close()

fmt.Println("Intercepting TLS traffic... (requires root)")
fmt.Println("Run: curl https://example.com in another terminal")

sig := make(chan os.Signal, 1)
signal.Notify(sig, os.Interrupt, syscall.SIGTERM)
go func() { ←sig; rd.Close() }()

var event SslEvent
for {
    record, err := rd.Read()
    if err ≠ nil {
        if ringbuf.IsClosed(err) { return }
        continue
    }
    if err := binary.Read(bytes.NewBuffer(record.RawSample), binary.LittleEndiar
        continue
    }
    direction := "WRITE"
    if event.Direction == 1 { direction = "READ " }

    payload := string(bytes.TrimRight(event.Payload[:event.Len], "\x00"))
```

```
        // Print first 120 chars to avoid flooding terminal
        if len(payload) > 120 { payload = payload[:120] + "..." }
        fmt.Printf("[PID %d] TLS %s %3d bytes: %q\n", event.PID, direction, event.Le
    }
}
```

---

8. Running the Lab

Phase 1: Compile & Run

```
cd series/series-04/source-code
make
sudo ./loader
```

Phase 2: Generate HTTPS Traffic in Another Terminal

```
curl https://httpbin.org/get
curl -X POST https://httpbin.org/post -d '{"user":"admin","pass":"secret"}'
python3 -c "import requests; requests.get('https://google.com')"
```

Phase 3: Observe the Intercepted Plaintext

```
[PID 9124] TLS WRITE 87 bytes: "GET /get HTTP/1.1\r\nHost: httpbin.org\r\nUser-Agent
[PID 9124] TLS READ  512 bytes: "HTTP/1.1 200 OK\r\nContent-Type: application/json\r
[PID 9125] TLS WRITE 68 bytes: "POST /post HTTP/1.1\r\n...\r\n{\"user\":\"admin\",\"
```

The sensitive POST body with the password is fully visible — before it was ever encrypted.

---

9. Debugging

```
# Check which processes have libssl loaded
lsof | grep libssl

# Verify the symbol exists at the expected offset
nm -D /usr/lib/x86_64-linux-gnu/libssl.so.3 | grep -E "SSL_write|SSL_read"

# If using a Go HTTPS server (uses crypto/tls, not libssl):
# Go TLS is implemented natively, not via libssl.
# For Go, hook the internal function:
# go.crypto/tls.(*Conn).Write
```

---

10. Common Mistakes & Verifier Errors

Error: `uprobe attach: no such symbol`

Cause: The application uses BoringSSL (Android, Chrome), LibreSSL (OpenBSD), or Go's native TLS — none of which expose `SSL_write`. **Fix**: Use `nm -D <libpath>` to find the actual function names in the target library.

Common Mistake: Reading SSL_read buffer on entry (not exit)

```
// WRONG — buffer is EMPTY on SSL_read entry!
SEC("uprobe/SSL_read")
int bad_read(struct pt_regs *ctx) {
    void *buf = (void *)PT_REGS_PARM2(ctx);
    bpf_probe_read_user(out, 512, buf); // reads garbage — data not yet populated
}
```

Fix: Save the buffer pointer on entry, read the buffer on exit (uretprobe) after the function has populated it.

Common Mistake: Stack overflow with large payload buffer

The BPF stack is 512 bytes. A 512-byte payload struct plus overhead exceeds it. **Fix**: Always use `bpf_ringbuf_reserve()` to write directly into ring buffer memory — the ring buffer is not on the BPF stack.

---

11. Performance Impact

Approach	App Slowdown	Visibility
OpenSSL SSLKEYLOGFILE	0%	Only with key export support
`strace` (ptrace)	**100–1000x**	Full
eBPF Uprobe	**< 1%**	**Full**

---

12. Summary

- Uprobes attach to user-space library functions with sub-microsecond overhead.
- `PT_REGS_PARM` macros extract function arguments from CPU registers via the System V ABI.
- For `SSL_write`: the plaintext buffer is readable on function **entry**.
- For `SSL_read`: the plaintext buffer is readable on function **exit** (uretprobe).
- `bpf_probe_read_user()` safely copies data from user-space memory into the BPF program.
- Ring Buffers are preferred over Perf Event Arrays for ordered, zero-copy event streaming.

---

13. Further Reading

- **Pixie TLS Tracing**: Pixie Blog — Debugging with eBPF
- **Cilium Layer 7 Visibility**: Cilium HTTP/gRPC Visibility
- **Linux Uprobe Internals**: Kernel Uprobe Documentation

Technical Deep-Dive Q&A: Uprobe-Based SSL/TLS Interception

This guide contains detailed technical questions for senior eBPF, security, and APM engineering roles.

---

1. Beginner Level

Q1. What is a Uprobe? How does it differ from a Kprobe?

Answer:

- **Kprobe** attaches to a **kernel-space** function by patching the kernel's instruction stream with an `int3` breakpoint. It requires no symbol files beyond `/proc/kallsyms`.
- **Uprobe** attaches to a **user-space** function inside a shared library or binary. The kernel patches the user-space ELF instruction at the specified symbol offset with a breakpoint. When any process executing that code hits the breakpoint, the eBPF program fires in kernel context with the ability to read the process's user-space memory.

Key difference: A single kprobe fires for all processes using that kernel function. A single uprobe fires for all processes that have mapped the target library (`/usr/lib/x86_64-linux-gnu/libssl.so.3`) — which is exactly what we want for system-wide TLS auditing.

Q2. Why can we read plaintext data from `SSL_write()` even though the traffic is encrypted on the wire?

Answer: Encryption is a sequential process:

1. Your application calls `SSL_write(ssl, buf, len)` with plaintext in `buf`.
2. OpenSSL reads `buf`, runs AES-GCM encryption, produces ciphertext.
3. Ciphertext is sent over the socket.

Between steps 1 and 2, the plaintext exists in user-space memory pointed to by `buf`. Our uprobe fires at **step 1** — the exact instruction where `SSL_write` begins executing. At this point,

`PT_REGS_PARM2(ctx)` points directly to the unencrypted buffer. We copy it with `bpf_probe_read_user()` before OpenSSL ever touches it.

This is not a vulnerability — it requires `CAP_BPF` (root or equivalent) to attach uprobes. It is the same principle used by Pixie, Datadog APM, and Cilium for zero-code HTTP visibility.

---

2. Intermediate Level

Q3. How does `bpf_probe_read_user()` differ from `bpf_probe_read()`? Why is the distinction critical for uprobes?

Answer: Both helpers safely copy memory into a kernel buffer, but they target different address spaces:

- `bpf_probe_read()` reads from **kernel virtual address space**. If you pass it a user-space pointer, it reads from whatever is mapped at that address in kernel space — which is either unmapped (fault) or completely wrong data.
- `bpf_probe_read_user()` explicitly reads from **the current process's user-space virtual address space**. It temporarily enables page fault handling during the copy, handles paged-out memory correctly, and ensures address validation.

For uprobes, `ctx→args[1]` (the `buf` pointer from `SSL_write`) is a user-space address. Using `bpf_probe_read()` instead of `bpf_probe_read_user()` will either crash or silently return garbage. This is a **silent correctness bug** — the verifier does NOT catch this mistake.

Q4. How do you find the exact byte offset of `SSL_write` inside `libssl.so` to attach a uprobe?

Answer: Uprobes attach by specifying: the **path to the ELF binary**, the **symbol offset** within that file (not a virtual address — a file offset relative to the binary's load base).

```
# Method 1: Use bpftool for uprobe-ready offset
sudo bpftool perf list | grep SSL_write

# Method 2: Calculate manually
LIBSSL=$(ldconfig -p | grep libssl | head -1 | awk '{print $4}')
readelf -Ws "$LIBSSL" | grep SSL_write
# Output: 0×0000000000048290 FUNC GLOBAL DEFAULT  13 SSL_write@@OPENSSL_3_0_0
# Offset = 0×48290

# Method 3: objdump symbol table
objdump -T /usr/lib/x86_64-linux-gnu/libssl.so.3 | grep " SSL_write"
```

The cilium/ebpf `link.Uprobe()` function takes this offset as `Offset: 0×48290`. The offset is different for every version of OpenSSL — production tools like Pixie maintain a database of offsets per version.

Q5. Explain what a Uretprobe is and why it is needed for `SSL_read()` but not for `SSL_write()`.

Answer:

- **Uprobe** fires at function **entry** — giving access to input arguments.
- **Uretprobe** fires at function **return** — giving access to the return value AND the output buffers that were filled during the call.

For `SSL_write(ssl, buf, len)`: the plaintext is in `buf` at **entry**. We use an uprobe at entry and read `buf` immediately.

For `SSL_read(ssl, buf, len)`: at **entry**, `buf` is an empty output buffer — OpenSSL hasn't written the decrypted data yet. The data is written INTO `buf` during the function execution, and is only valid **after the function returns**. We need a uretprobe to fire after `SSL_read` completes, at which point `buf` contains the decrypted plaintext.

The challenge: a uretprobe cannot directly access the original function arguments (they are gone from registers). Solution: save `{pid, buf_ptr}` in a BPF hash map at the uprobe entry, look it up at the uretprobe using the PID as key.

---

3. Advanced Level

Q6. SSL_write is different across OpenSSL 1.x and 3.x. How would you make your uprobe binary-portable across versions?

Answer: Rather than hardcoding the function offset, production tools use one of three strategies:

1. **Symbol name lookup at load time**: Use `link.Uprobe()` with `Symbol: "SSL_write"` instead of `Offset`. The cilium/ebpf library resolves the symbol to an offset at runtime using the ELF symbol table of the currently loaded library. This automatically handles version differences as long as the function name is stable.

2. **Version-specific offset database**: Pixie maintains a JSON database mapping `(library_path_hash, symbol_name)` → `offset` for every common version. The agent downloads the right entry at startup.

3. **USDT (Userspace Statically Defined Tracepoints)**: OpenSSL 3.x adds USDT probes (`provider:openssl ssl_write_ex_start`). These are stable ABI entries similar to kernel tracepoints — the offset is defined by the application developer and guaranteed not to change. Use `link.USDT()` in cilium/ebpf.

---

4. System Design & Scenario-Based

Q7. Design a production zero-code HTTP request tracer that captures request/response pairs for all services in a Kubernetes cluster without modifying any application.

Answer:

```
Architecture:
  DaemonSet: ebpf-tracer (one pod per node, --privileged)
    │
    ├─ Uprobe: libssl.so SSL_write → capture outgoing HTTP request body
    ├─ Uretprobe: libssl.so SSL_read → capture incoming HTTP response body
    │
    ├─ BPF Maps:
    │    active_reads  HASH {pid → buf_ptr}   (correlate entry → return)
    │    events        PERF_EVENT_ARRAY        (stream data to Go agent)
    │
    └─ Go Agent:
         - Parse HTTP/1.1 from raw bytes (look for "HTTP/1.1", "POST /", headers)
         - Correlate request → response by PID + sequence number
         - Emit as OpenTelemetry spans to Jaeger/Tempo
         - Label with pod name from /proc/<pid>/cgroup
```

Key engineering decisions:

1. **Attach to the process namespace**: Use `/proc/<pid>/root` to find the actual library path inside each container (containers may have their own libssl).
2. **Buffer size limit**: Cap captured data at 4096 bytes per event. Full HTTP bodies can be 10MB+ — streaming them all would overwhelm the perf buffer.
3. **HTTP/2 support**: gRPC uses HTTP/2, which multiplexes streams. You need to track the stream ID from the HPACK frame header to correlate requests and responses.

---

5. Troubleshooting & Debugging

Q8. Your uprobe is attached but captures zero events, even though `curl https://example.com` is clearly making TLS connections. List 4 likely causes.

Answer:

1. **Wrong library path**: The process may use a different OpenSSL build. Check:

```
sudo cat /proc/$(pgrep curl)/maps | grep ssl
```

If it shows `/snap/curl/current/usr/lib/libssl.so.3` instead of `/usr/lib/...`, your uprobe is attached to the wrong file.

2. **Statically linked binary**: Some Go binaries (e.g., compiled with `CGO_ENABLED=0`) use Go's own TLS implementation (`crypto/tls`), not OpenSSL. There is no `SSL_write` symbol to hook — you'd need to hook `crypto/tls.(*Conn).Write` instead.

3. **Symbol stripped from binary**: Production builds often strip debug symbols. Check:

```
nm /usr/lib/x86_64-linux-gnu/libssl.so.3 | grep SSL_write
```

   If empty, the library is stripped. Use `readelf -Wd` to find the dynamic symbol table (`.dynsym`) which is required for shared library operation and cannot be stripped.

4. **PID namespace mismatch**: The uprobe path `/usr/lib/.../libssl.so.3` is relative to the **host** filesystem. A container may have its own libssl at a different inode inside its overlay filesystem. Use the container's `/proc/<pid>/root/usr/lib/...` path when attaching.

Hands-on Lab & Homework Assignments

1. Practical Exercises & Research Tasks

Task 1: Find SSL_write in the ELF Binary

Before writing eBPF code, understand what you are hooking:

```
# Find the OpenSSL library on your system
LIBSSL=$(ldconfig -p | grep "libssl.so" | tail -1 | awk '{print $4}')
echo "Library: $LIBSSL"

# List SSL_write and SSL_read symbols with their offsets
readelf -Ws "$LIBSSL" | grep -E "SSL_write|SSL_read"

# Verify dynamic symbol (cannot be stripped — required for linking)
readelf -Wd "$LIBSSL" | grep -E "SSL_write|SSL_read"

# Disassemble the first 20 instructions of SSL_write
objdump -d "$LIBSSL" | grep -A 20 "<SSL_write>"
```

- What is the byte offset of `SSL_write` in your installed version?
- Does the offset change after an `apt upgrade openssl`? Why does this matter for production uprobe agents?

Task 2: Inspect the Uprobe with bpftool

After running `sudo ./loader`:

```
# List perf events including uprobes
sudo bpftool perf list

# Find your uprobe programs
sudo bpftool prog list | grep -E "ssl_write|ssl_read"

# Check run counts — should increase on each TLS connection
sudo bpftool prog show name capture_ssl_write --pretty
```

Run `curl https://httpbin.org/get` in another terminal and verify `run_cnt` increments.

---

2. Coding Assignment: Capture SSL_read (Decrypted Responses)

Objective: Extend the auditor to also capture the decrypted response from `SSL_read`.

Requirements:

1. Add an entry map to save the buffer pointer at `SSL_read` entry:

```
struct ssl_read_args {
    __u64 buf_ptr;  // pointer to output buffer
    __u32 len;      // max length to read
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 1024);
    __type(key, __u32);                // PID
    __type(value, struct ssl_read_args);
} active_reads SEC(".maps");
```

2. Add an uprobe at `SSL_read` entry that saves `{buf_ptr, len}` keyed by PID.
3. Add a **uretprobe** at `SSL_read` return that:
   - Looks up `active_reads[pid]`
   - Checks `PT_REGS_RC(ctx)` (return value = bytes actually read, or -1 on error)
   - If return value > 0, reads `buf_ptr` with `bpf_probe_read_user_str`

- Submits a "RESPONSE" event to the perf buffer

4. In Go, display events with direction:

```
[PID=9124] WRITE → POST /api/login HTTP/1.1
```

Host: auth.example.com [PID=9124] READ ← HTTP/1.1 200 OK {"token":"eyJhbGc..."}

```
---

## 3. Mini-Project: HTTP Request Parser
**Objective**: Parse the captured SSL bytes as HTTP/1.1 and extract the method, path

### Design in Go:
```go
type HTTPEvent struct {
 Direction string  // "REQUEST" or "RESPONSE"
 Method    string  // "GET", "POST", etc.
 Path      string  // "/api/v1/users"
 Status    int     // 200, 404, 500
 Host      string  // "api.example.com"
}

func parseHTTP(data []byte, dir string) *HTTPEvent {
 lines := strings.Split(string(data), "\r\n")
 if len(lines) == 0 { return nil }

 ev := &HTTPEvent{Direction: dir}
 if dir == "REQUEST" {
     // "POST /api/login HTTP/1.1"
     parts := strings.Fields(lines[0])
     if len(parts) >= 2 { ev.Method, ev.Path = parts[0], parts[1] }
     for _, l := range lines[1:] {
         if strings.HasPrefix(l, "Host: ") { ev.Host = l[6:] }
     }
 } else {
     // "HTTP/1.1 200 OK"
     parts := strings.Fields(lines[0])
     if len(parts) >= 2 { fmt.Sscanf(parts[1], "%d", &ev.Status) }
 }
 return ev
}
```

Display a clean request/response log:

```
2024-01-15 10:23:45  POST  /api/login          → auth.example.com [PID=9821]
2024-01-15 10:23:45  ←     200 OK                                [PID=9821]
2024-01-15 10:23:46  GET   /api/user/profile   → api.example.com  [PID=9821]
2024-01-15 10:23:46  ←     200 OK                                [PID=9821]
```
```

4. Advanced Challenge: Multi-Process TLS Monitor Dashboard

Build a terminal dashboard (using Go's `termbox-go` or simple ANSI codes) that shows:

```
┌─── Active TLS Sessions ──────────────────────────────────────────┐
│ PID    COMM       HOST                  Requests  RX Bytes  TX  │
│ 9821   curl       api.example.com             12    45.2KB  12.1KB │
│ 8734   python3    oauth2.google.com            3     8.4KB   2.3KB │
│ 11230  node       api.stripe.com              88   320.1KB  45.6KB │
└──────────────────────────────────────────────────────────────────┘
```

Update the table every 1 second. Track:

- Total requests per PID (count of `SSL_write` events)
- Total bytes sent (sum of `len` args from `SSL_write`)
- Total bytes received (sum of return values from `SSL_read`)

Write a short analysis: Which process is making the most TLS connections? Does its traffic pattern (request size vs response size ratio) suggest it is a REST API client or a streaming client?